

Bidirectional Alignment as Situated Repair: Linking Configuration and Runtime

M. Antonietta Grasso, Jisun Park, Jutta Willamowski

NAVER LABS Europe

Grenoble, France

firstname.surname@naverlabs.com, jisun.p@naverlabs.com

Abstract

In bidirectional human–AI alignment, alignment is conceptualized as a dynamic process of reciprocal adjustment between people and AI systems across time and contexts of use. While this agenda often foregrounds broad societal values, we focus on a concrete value that helps realize them in practice: repairability, understood as minimizing struggle by making breakdowns detectable, correctable, and accountable across stakeholders. We argue that repair work cannot be fully specified or reliably evaluated in advance because it is shaped by the setting—including workflows, operational constraints, available evidence (e.g., monitoring/logs), and participants’ roles and authority—and by the consequences of mistaken or premature risky actions. We propose two complementary layers for alignment in use: configuration-time governance to articulate AI requirements as a coherent set for stakeholder review, and setting-dependent runtime orchestration that may operate in trace-only mode (to support verification and learning) or enforce runtime repair support. We also propose a lightweight evaluation approach that (1) defines the setting and stakeholders, (2) extracts repair episodes from logs, and (3) evaluates detectability, correctability, and accountability from episode outcomes. We illustrate the approach with a coffee-ordering chatbot facing misunderstandings, underspecified requests, and iterative clarification.

Keywords

human–AI alignment; evaluation; repair work; deployment; logs; conversational agents

ACM Reference Format:

M. Antonietta Grasso, Jisun Park, Jutta Willamowski. 2026. Bidirectional Alignment as Situated Repair: Linking Configuration and Runtime. In . ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Alignment as Repair-in-Context

Although alignment is often discussed in terms of broad societal values, our focus is intentionally concrete. In everyday deployments, a central value is that people should not have to struggle to make a system do what they mean. We treat this as a value of *practical agency*: the ability to steer, correct, and contest system behavior

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference’17, Washington, DC, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

without excessive effort, confusion, or hidden work. In deployed services, this value is realized (or undermined) through *repairability*—whether breakdowns can be detected, clarified, and corrected; whether responsibility can be handed to the right stakeholder; and whether the system avoids premature executions while uncertainty remains. This framing also connects to “bigger” concerns, because the distribution of repair burden affects fairness and work practices, and failure to manage repair near execution boundaries can create safety and accountability risks.

Building on this view, we start from a pragmatic observation: when AI systems are deployed as services, they do not remain “aligned” because a model is intrinsically correct, nor because a fixed set of rules perfectly anticipates the world. They remain aligned when the service is *repairable*—when breakdowns are detectable, when there are safe ways to clarify and correct, when executions are legible and reversible, and when responsibility can be handed to the right stakeholder at the right time. This view has a long lineage. In conversation analysis, repair is a core organizational mechanism through which participants restore mutual understanding, with a strong preference for self-correction [5]. In psycholinguistic and HCI traditions, this connects to *grounding*: the interactive work required to establish what is being referred to and what is being executed [2]. In conversational systems, breakdowns and repair strategies have been studied as a central determinant of user experience and resilience, including preferences for different repair moves when chatbots fail to understand [1].

Recent work on *bidirectional* human–AI alignment makes this point explicit by separating two intertwined processes: aligning AI with humans (through human specifications used to steer and customize systems) and aligning humans with AI (supporting people to understand, critique, collaborate with, and adapt to AI) [7]. This framing also broadens who counts as “human in the loop.” Rather than treating people as mere end-users, we distinguish stakeholders by their role in repair: *repair actors* who detect and resolve breakdowns, *execution authorities* who can authorize binding actions, and *impacted parties* who bear downstream benefits or harms [7]. This matters in deployment because repairability depends on whether these roles have clear decision rights, workable escalation paths, and predictable ways to prevent or undo premature executions.

A CSCW lens extends this account beyond AI interaction to how breakdowns are handled in real work. In real work settings, repair is distributed across people, tools, and organizational arrangements: breakdowns surface through logs, tickets, and handoffs, and recovery depends on roles, authority, and accountability. Classic CSCW arguments emphasize that cooperative systems rely on situated action rather than complete plans [9], and that keeping work going often requires *articulation work*—the coordination, negotiation, and

exception handling that makes tasks feasible under real constraints [6, 8]. In this view, “repair” is not only fixing misunderstandings in dialogue; it also includes diagnosing failures, routing responsibility, deciding what evidence counts, and restoring safe operation.

Repair becomes especially important in many AI services because it captures two practical costs: the cost of reaching mutual understanding, and the cost of addressing breakdowns when understanding fails. When inputs can trigger real work—updating records, placing orders, changing configurations, or initiating recovery steps—misunderstandings are not only frustrating; they can create rework, delays, and accountability risks.

Second, these costs are rarely borne by a single person because deployments are inherently multi-stakeholder. A customer may care about speed and convenience, store staff may care about predictable handoffs and minimal rework, and an operations team may need traceable diagnosis and safe rollback. “Alignment” in this sense is not a single preference function. It is a situated achievement that must hold across stakeholders and over time.

This creates a gap: configuration alone cannot anticipate situated breakdowns, and runtime adaptation alone is unaccountable without explicit executions and evidence. This motivates our central claim: to support bidirectional alignment realistically, we need *both* configuration-time governance and runtime mediation, and we should evaluate them together in context. Configuration is where stakeholders define what the service is for, what it must not do, which actions require confirmation, and how escalation and accountability are handled. Runtime is where those executions meet uncertainty in use—underspecified requests, ambiguous evidence, and evolving situations—and the service must remain repairable rather than “charging ahead.” Crucially, runtime is also where evidence is produced: logs that capture repair episodes and execution decisions, rather than only isolated inputs and outputs.

2 Configuration and Runtime Are Complementary

Configuration-time governance matters because it is where alignment requirements must be made explicit before deployment. In our principle-set workbench, we studied how stakeholders articulate and revise organizational, cognitive, and emotional requirements as a coherent set of principles, and how coherence issues (overlap, contradiction, gaps) arise during authoring [3]. While this work did not connect principles to a specific production service, it motivates a practical implication for deployment: principles should not remain only as plain text injected into prompts. Plain-language principles are valuable for deliberation and communication, but production services require predictable behavior at execution boundaries. Before deployment, we propose operationalizing each principle with a small checklist: *when* it applies, *who* acts (and who can authorize, if relevant), and *what* should happen; some principles also specify a *phase* (e.g., before execution, after correction, on escalation). This lightweight structure keeps principles readable for stakeholders while making them easier to enact through runtime orchestration.

This does not require a full policy language; it is a practical way to ensure that stakeholder intent can be enacted reliably.

At the same time, configuration-only approaches remain insufficient. The first reason is empirical: deployments reveal breakdowns

that were not anticipated, and repair practices are shaped by the setting and the distribution of roles. The second reason is structural: in persistent-state services, the cost of error depends on *when* a decision becomes binding. This is why runtime mediation is needed. We therefore assume a runtime orchestration layer that mediates between conversational interpretation and consequential service action. Concretely, it consumes user turns together with UI/service events, maintains a compact turn context over the current service object and recent interaction history, and determines what kind of move is authorized at that moment. While uncertainty remains, it can keep updates in draft, request clarification or disambiguation, present previews, require explicit confirmation before commitment-bearing actions, and support rollback/undo or escalation when needed. In this way, the runtime layer does not simply produce outputs; it governs the transition from interpretation to commitment and records the resulting decisions and outcomes as traceable evidence for later review.

A useful way to summarize the relationship is this: configuration is the rulebook (principle set) for authorized execution, extending what described in [3]. It specifies the service requirements and which high-impact actions must pass explicit checkpoints, in a form stakeholders can review. Runtime orchestration determines what the service is authorized to do next—for example, ask a targeted clarification, preserve a draft, preview a change, apply a reversible update, escalate, or proceed to commitment—given the current interaction state, the relevant service state, and the risk of acting now. If configuration is too weak, runtime has little guidance and execution gating becomes inconsistent. If runtime is too weak, configuration must anticipate every edge case, becomes brittle, and repair gets pushed onto people after high-impact actions have already been executed. Bidirectional alignment is the ongoing coupling between the two: runtime traces reveal how breakdowns and repairs actually unfold in the deployed setting; governance updates then refine authorization thresholds, checkpoints, and allowed interventions.

If alignment is sustained by repair, then evaluation should focus on repair episodes in deployment. We propose a deliberately lightweight evaluation scaffold that teams can adopt using ordinary logs, without heavy instrumentation or complex experimentation. The scaffold treats a deployment setting—with stakeholders and *authorized-execution* boundaries—as the unit of analysis, rather than a model in the abstract. It also distinguishes two practical concerns that often get conflated: (i) high-frequency breakdowns that primarily create *interaction burden* (extra turns, retries, handoffs, articulation work), and (ii) rarer *execution failures* where the service takes, or nudges toward, a high-cost or irreversible authorized action before uncertainty is resolved. Teams fill the sheet at configuration time, then revisit it after deployment using logs to identify repair episodes and update authorization rules and checkpoints.

Operationalizing Situated Repair Evaluation (SRE) with orchestrator logs. An SRE sheet (illustrated in Fig.1) uses the logs of a runtime orchestrator as its measurement layer. We assume an orchestration layer that sits between conversational interpretation and commitment-bearing service action. At each turn, it receives the user utterance together with relevant UI or service events, maintains a typed turn context over the current object state and

Situated Repair Evaluation (SRE) Sheet

- 1) **Setting and authorized execution.** What is the service? What counts as a high-cost or irreversible *authorized execution* in this setting?
- 2) **Stakeholders (repair actors / execution authorities / impacted parties).** Who detects breakdowns? Who can authorize execution or rollback/undo? Who is affected downstream? Who bears routine repair effort vs. incident repair work?
- 3) **Expected breakdowns.** What are the most common breakdown types, and what are rare but high-severity near-execution failures to watch for?
- 4) **Repair path.** How should the service clarify, incorporate corrections, roll back/undo, or escalate *before authorized execution*? Where are explicit checkpoints required?
- 5) **Evidence in orchestrator logs.** Which trace events indicate breakdown and repair, and what measures will be used to assess outcomes?

Figure 1: A lightweight scaffold for evaluating alignment as repair-in-context: it links configuration requirements to runtime traces, and separates routine interaction burden from rare, high-consequence execution failures.

recent interaction history, and authorizes an interaction move under service policy. Typical authorized moves include asking for clarification, highlighting or previewing alternatives, applying or revising a draft update, requesting confirmation, committing a change, or supporting rollback/undo and escalation. Because the orchestrator records the context, the move selected, whether any checkpoint or authorization occurred, and the resulting state transition, the trace makes repair episodes visible for evaluation. For each repair episode (a correction, clarification, rollback/undo, or escalation), we read three things from the trace: (i) the final state (draft or *executed*), (ii) the repair moves taken, and (iii) whether a checkpoint/authorization occurred before any authorized execution. This yields three measures: *repair success* = episode ends in an acceptable state; *repair cost* = time and interaction work to get there (turns, retries, handoffs/escalations); *execution safety* = authorized executions occur only after checkpoints and not while repair is ongoing (with rollback/undo and escalation available when needed). These three measures instantiate three repairability criteria: *detectability* is reflected in whether breakdowns surface as identifiable episodes in orchestrator traces; *correctability* is captured by repair success and repair cost; and *accountability* is captured by execution safety through explicit checkpoints/authorizations, rollback/escalation traces, and role-specific decision rights. The SRE sheet provides the setting-specific definitions (what counts as an acceptable outcome, what counts as authorized execution, and what checkpoints/authorizations are required), so the same trace can be scored consistently across releases.

This bidirectional perspective also clarifies why deployment-centered evaluation cannot be reduced to a single score. The bidirectional alignment approach emphasizes that evaluation and oversight span multiple targets—from model behavior and policy compliance to real-world impacts—and that practical evaluation fails in characteristic ways: rare but severe failures are “needles in a haystack,” and different evaluation setups can inadvertently reward undesirable behavior (e.g., guessing rather than appropriately expressing uncertainty) [4]. These observations motivate an evaluation stance that is closer to CSCW accounts of breakdown and recovery. Rather than assuming we can pre-enumerate all relevant failure modes, we evaluate whether the system supports *situated recovery* when breakdowns occur, who bears the burden of repair, and whether premature authorized executions are avoided while uncertainty remains. In our proposal, logs provide the connective tissue between configuration and runtime: they make repair episodes visible, enable comparison across stakeholders and settings, and provide evidence for revising governance requirements over time.

While we emphasize execution boundaries because they make costs and accountability visible, execution gating is not the whole of alignment in our framing. Repairability also concerns (i) *interpretation repair*—how the system grounds references, asks targeted clarifications, and incorporates corrections without losing progress; (ii) *expectation repair*—how the system makes its understanding and next actions legible so stakeholders can contest or redirect; and (iii) *responsibility repair*—how breakdowns are routed through escalation paths and how accountability is maintained across roles. Execution checkpoints are one concrete enforcement point within this broader repair space, but evaluation should capture the wider pattern of situated recovery across stakeholders.

The evaluation itself is centered on three families of measures that remain meaningful across different services. The first is *repair success*: when a breakdown occurs, does the interaction return to a satisfactory trajectory? This can be operationalized as reaching a correct final state (e.g., the right order contents, payment status, and fulfillment choice). The second is *repair cost*: how much time, interaction, or human effort is required to recover. In practice, this can be approximated with time-to-resolution, number of turns, number of retries, and frequency of escalation to a human owner. The third is *execution safety*: does the service avoid premature executions when uncertainty is high? In the runtime orchestration framing, this is visible through draft-versus-execution behavior, explicit checkpoints, and whether corrections are handled while the system still preserves reversibility.

Methodologically, the workflow is intentionally lightweight. Teams sample repair episodes from logs using simple triggers such as: clarifying questions; user corrections; explicit undo/rollback events; escalations; and state diffs that indicate a proposed update being executed. Quantitative summaries provide a baseline across time windows and releases. Then, rather than conducting large-scale annotation, the team performs a small qualitative review of strategically selected episodes (for example, all escalations plus a random slice of successful repairs). The purpose of the qualitative pass is not to build a universal taxonomy; it is to understand whether the repair path worked for the relevant stakeholders in this particular setting, and to identify configuration changes that

could prevent recurring breakdowns or reduce repair cost. This realizes a simple feedback loop: runtime traces identify repair realities; governance evolves accordingly.

Finally, this evaluation loop can also feed the classic ML improvement cycle. By organizing deployment data as repair episodes (triggers, repair moves, who authorized, and outcomes), the same runtime evidence used for evaluation becomes structured training signal for subsequent iterations (e.g., improving clarification strategies in ordering), without relying on raw interaction logs alone.

3 A Setting: Coffee Orders

We briefly show how the evaluation logic applies in a concrete setting where pace, ambiguity, and execution boundaries are easy to see.

In a coffee-ordering chatbot, repair often appears as routine interactional work: missing attributes (size, temperature, milk), ambiguous references (“the usual” or “that one”), and changes of mind. Here, repair actors include customers and store staff; the customer typically authorizes checkout, while exceptions (e.g., substitutions or refunds) may be governed by staff workflow or policy. The central execution boundary is checkout/payment: once the system executes, recovery becomes costly. Runtime mediation is therefore essential to keep the interaction repairable: preserve progress while keeping the order in draft, incorporate corrections without losing context, and clarify underspecified requests rather than guess. Evaluation can treat each session as a sequence of repair episodes and phase transitions: how often repair was needed, whether it succeeded, how much effort it took (e.g., turns or time), and whether binding steps were appropriately authorized. The same measures (repair success, repair cost, and execution safety/accountability) can be computed, but interpreted through the setting: “cost” may appear as abandonment or staff intervention, while “safety” concerns incorrect payment or fulfillment.

In this setting, the evaluation emphasis is intentionally not on a single correctness score. What matters is whether the service supports repair that is appropriate to the stakeholders and to the execution boundary of the setting, and whether runtime evidence supports learning and governance updates over time. This is where configuration-time principle work and runtime orchestration become mutually reinforcing: configuration clarifies execution and escalation expectations; runtime supports repairability and produces traces; and evaluation uses those traces to refine governance.

4 Conclusion

Our proposal does not assume a particular learning machinery. Fine-tuning, preference optimization, and prompt-based principle enforcement can reduce the frequency of breakdowns and improve default behavior, but they are not sufficient on their own in deployed services where actions become binding: they do not reliably enforce safe authorized execution at runtime or ensure role-appropriate escalation. We therefore treat learning and orchestration as complementary. In our framing, a runtime orchestrator enforces structured authorized execution at execution boundaries by distinguishing proposed, drafted, and committed changes; requiring explicit checkpoints before high-cost actions; and preserving rollback, undo, or

escalation paths when uncertainty remains. It also produces logs that make repair episodes explicit by recording turn context, selected repair or execution moves, authorization events, and resulting state transitions. These traces support learning by organizing deployment data into repair episodes with outcomes and effort: successful repairs (including those that are still costly) provide exemplars for improving default strategies (e.g., better clarifications and fewer retries), while repeated failures and escalations highlight setting-specific gaps to address. At the same time, we retain explicit gating for high-cost actions: if an authorized execution happens prematurely, the response is to strengthen checkpoints and authorization rules at the execution boundary (and enforce draft/rollback/escalation), rather than relying on the model to self-regulate based on model confidence alone. In short, our approach learns repair paths while strengthening the gating of binding actions.

For the BiAlign workshop, our contribution is a lightweight framing and an evaluation scaffold. We suggest treating bidirectional alignment as a repair process that links configuration and runtime in deployment. We offer a SRE template that anchors evaluation in the setting, its stakeholders, and its execution boundaries, and we propose a log-based workflow that teams can run with low overhead, especially when an orchestrator is in place. We expect this framing to be useful in workshop activities because it is concrete: participants can instantiate the sheet for their own services, compare what “authorized execution” means across domains, and discuss how repair paths and evaluation signals change with stakeholder structure. More broadly, it shifts alignment discussions from model properties to sociotechnical practice, while remaining operational and measurable through repairability outcomes.

References

- [1] Zahra Ashktorab, Mohit Jain, Q. Vera Liao, and Justin D. Weisz. 2019. Resilient Chatbots: Repair Strategy Preferences for Conversational Breakdowns. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems (CHI '19)*. Association for Computing Machinery, New York, NY, USA, Article 254, 12 pages. doi:10.1145/3290605.3300484
- [2] Herbert H. Clark and Susan E. Brennan. 1991. Grounding in Communication. In *Perspectives on Socially Shared Cognition*, Lauren B. Resnick, John M. Levine, and Stephanie D. Teasley (Eds.). American Psychological Association, Washington, DC, USA, 127–149. <https://philpapers.org/rec/CLABG-2>
- [3] Antonietta Maria Grasso, Jisun Park, Jutta Katharina Willamowski, Laurent Besacier, and Jos Rozen. 2026. Surfacing Governing Principles for Chatbots: A Workbench and Comparative Study. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems (CHI '26)*. 14 pages. doi:10.1145/3772318.3790612
- [4] Adam Tauman Kalai. 2025. Evaluation and Oversight of today’s AI (Tutorial slides). Part IV of NeurIPS 2025 Human-AI Alignment tutorial slides.
- [5] Emanuel A. Schegloff, Gail Jefferson, and Harvey Sacks. 1977. The Preference for Self-Correction in the Organization of Repair in Conversation. *Language* 53, 2 (1977), 361–382. <https://www.jstor.org/stable/413107>
- [6] Kjeld Schmidt and Carla Simone. 1996. Coordination mechanisms: Towards a conceptual foundation of CSCW systems design. *Computer Supported Cooperative Work (CSCW)* 5, 2–3 (1996), 155–200. doi:10.1007/BF00133655
- [7] Hua Shen, Tiffany Knearem, Reshmi Ghosh, Kenan Alkiek, Kundan Krishna, Yachuan Liu, Ziqiao Ma, Savvas Petridis, Yi-Hao Peng, Li Qiwei, Sushrita Rakshit, Chenglei Si, Yutong Xie, Jeffrey P. Bigham, Frank Bentley, Joyce Chai, Zachary Lipton, Qiaozhu Mei, Rada Mihalcea, Michael Terry, Diyi Yang, Meredith Ringel Morris, Paul Resnick, and David Jurgens. 2024. Position: Towards Bidirectional Human-AI Alignment. arXiv:2406.09264 [cs.AI]
- [8] Anselm Strauss. 1985. Work and the Division of Labor. In *Social Organization of Medical Work*. University of Chicago Press, Chicago, IL, USA.
- [9] Lucy A. Suchman. 1987. *Plans and Situated Actions: The Problem of Human-Machine Communication*. Cambridge University Press, Cambridge, UK.